

Theory elements for the creation of builders. The goal of this article is to give some theoretic tools to study builder pointing.

Builders use nested tuples as arguments, the first question is then to understand at which condition a tuple can be considered valid for a rule. Then, it will be demonstrated that pointed tuple are converted by the algorithm to non pointed tuples. Finally, unpointing need to ensure that a non pointed tuple of size n (as a combinatorial object) has exactly n pointed tuple of size n mapped to it by the algorithm.

Some definitions elements:

- RuleName present in the UsainBoltz library are called non terminal.
- a python tuple is noted with a prefix Tp: $(1, 2)$ becomes $\text{Tp}(1, 2)$. Mathematical couples are noted normally.

Some aspects present in the real algorithms are simplified:

- there is a *Zero* rule which corresponds to the empty class. This class will be used instead of simplifying as done in the real algorithms.
- we assume that builders always take as argument tuples and return tuples. For iterated rules, it is assumed that lists are tuples. For other objects (which are not tuples nor lists), they are considered to be a singleton with a single element.

We first define sub-right-rules:

Definition 1 *A sub-right-rule (which can be read as a sub rule of the right of a production rule in a grammar) is defined by induction:*

Atom(), Epsilon(), Marker(name), RuleName(name) for name a character chain.

are sub-right-rules.

If $R, R_1, \dots, R_n$ are sub-right-rules,

$$\text{Union}(R_1, \dots, R_n), \text{Product}(R_1, \dots, R_n), \text{Seq}(R), \text{Set}(R), \text{Cycle}(R)$$

are sub-right-rules.

For R a sub-right-rule, we say that S another sub-right-rule appear in R if R can be obtained by induction with S as one of R base element.

Example 1 *$R = \text{Union}(\text{Product}(\text{Atom}(), \text{Epsilon}(), \text{Epsilon}()), \text{Atom}())$ is a sub-right-rule.*

$S_1 = \text{Product}(\text{Atom}(), \text{Epsilon}(), \text{Epsilon}())$ appears in R because $\text{Union}(S_1, \text{Atom}()) = R$.

$S_2 = \text{Product}(\text{Atom}(), \text{Epsilon}())$ doesn't appear in R because $\text{Product}(\text{Product}(\text{Atom}(), \text{Epsilon}()), \text{Epsilon}()) \neq R$ (je sais pas comment finir ce cas).

From this definition we can define a grammar.

Definition 2 A grammar G is a set of production rules where a production rule is a couple (A, R) noted $A \rightarrow R$. A is a RuleName called a non terminal of G and R is a sub-right-rule called a right-rule of G .

We say that a sub-right-rule S appears in G if it appears in a right-rule of G .

An additional condition on G to be considered a grammar is that all sub-right-rules A which are RuleNames also are non terminals of G and each non terminal G is only associated with a single right-rule.

We define the pointing algorithm on a sub-right-rule. Even though this code looks like python, there are pseudo-code elements. We could say this is pseudo-python.

```
#Some shortcuts are allowed: None-1 = None; (i-j) = max(0, i-j). Very useful for sizes constraints.
def Point(S: SubRightRule) -> SubRightRule:
    switch S:
        case Atom():
            return Atom()
        case Epsilon() | Marker(-) | Zero():
            return Zero()
        case Union(R1, ..., Rn):
            return Union(Point(R1), ..., Point(Rn))
        case Product(R1, ..., Rn):
            return Union(
                Product(Point(R1), R2, ..., Rn),
                ...,
                Product(R1, ..., Point(Rn))
            )
        case RuleName(name):
            return RuleName(name + ".P")
        case Set(R, leq = i, geq = j):
            return Product(R, Set(Point(R), leq = i-1, geq = j-1))
        case Cycle(R, leq = i, geq = j):
            return Product(R, Seq(Point(R), leq = i-1, geq = j-1))
        case Seq(R, leq = None, geq = None):
            return Union(Product(Seq(R, eq = 0), Point(R), Seq(R)))
        case Seq(R, leq = i, geq = j):
            #with i != None.
            return Union(
                Product(Seq(R, eq = 0), Point(R), Seq(R, leq = i-1, geq = j-1)),
                Product(Seq(R, eq = 1), Point(R), Seq(R, leq = i-2, geq = j-2)),
                ...,
                Product(Seq(R, eq = i-1), Point(R), Seq(R, leq = 0, geq = j-i))
            )
        case Seq(R, leq = None, geq = j):
            #with j != None
            return Union(
                Product(Seq(R, eq = 0), Point(R), Seq(R, leq = None, geq = j-1)),
                ...,
                Product(Seq(R, eq = j-2), Point(R), Seq(R, leq = None, geq = 1)),
                Product(Seq(R, geq = j-1), Point(R), Seq(R))
            )
```

Now we define the tuple unpointing algorithm.

```
def UnpointTp(t: tuple, S: SubRightRule) -> tuple:
    switch S:
        case Union(R1, ..., Rn):
            Tp(i, tu) = t #This can raise an error if t has the wrong format.
```

```

return Tp(i, Unpoint(Ri, tu))
case Product(R1, ..., Rn):
  Tp(i, Tp(t1, ..., tn)) = t
  return Tp(t1, ..., Unpoint(Ri, ti), ..., tn)
case RuleName(-):
  return t
case Cycle(R) | Set(R):
  Tp(t1, Tp(t2, ..., tn))
  return Tp(Unpoint(R, t1), t2, ..., tn)
case Seq(R):
  Tp(, Tp(Tp(t_left_1, ..., t_left_n), t_pointed, Tp(t_right_1, ..., t_right_m))) = t
  return Tp(t_left_1, ..., t_left_n, Unpoint(R, t_mid), t_right_1, ..., t_right_m)
case Epsilon() | Marker(-) | Zero():
  raise Exception("Error")

```

Definition 3 (Acceptable tuple for a sub-right-rule.) A context Γ is a couple (G, M) with G a grammar and M is a mapping from the set of non terminals in G to builders. For $A \rightarrow R$ a production rule in G , the builder $\mathcal{B} = M(A)$ is a function from the set of right-hand acceptable tuples for R to a set $E_{\mathcal{B}}$.

For Γ a context, S a sub-right-rule of G , t^v and t^b two tuples, we say that (t^v, t^b) is acceptable for S and we note $\Gamma : S \vdash (t^v, t^b)$ if this proposition can be deduced from the following inference rules:

$$\begin{array}{c}
\frac{}{\Gamma : \text{Epsilon}() \vdash (\text{Epsilon}(), \text{Epsilon}())} \text{Epsilon} \\
\\
\frac{}{\Gamma : \text{Atom}() \vdash (\text{Atom}(), \text{Atom}())} \text{Atom} \\
\\
\frac{}{\Gamma : \text{Marker}('a') \vdash (\text{Marker}('a'), \text{Marker}('a'))} \text{Marker} \\
\\
\frac{i \in [1; n] \quad \Gamma : S_i \vdash (t^v, t^b)}{\Gamma : S_1 + \dots + S_n \vdash (Tp(i, t^v), Tp(i, t^b))} \text{Union } n \geq 2 \\
\\
\frac{\Gamma : S_1 \vdash (t_1^v, t_1^b) \quad \dots \quad \Gamma : S_n \vdash (t_n^v, t_n^b)}{\Gamma : S_1 \times \dots \times S_n \vdash (Tp(t_1^v, \dots, t_n^v), Tp(t_1^b, \dots, t_n^b))} \text{Product } n \geq 2 \\
\\
\frac{\Gamma : S \vdash (t_1^v, t_1^b) \quad \dots \quad \Gamma : S \vdash (t_n^v, t_n^b)}{\Gamma : \text{Seq}(S) \vdash (Tp(t_1^v, \dots, t_n^v), Tp(t_1^b, \dots, t_n^b))} \text{Sequence} \\
\\
\frac{A \rightarrow R \in G \quad \Gamma : R \vdash (t^v, t^b)}{\Gamma : A \vdash (t^v, M(A)(t^b))} \text{Simplification}
\end{array}$$

The b in t^b stands for builder because t^b is obtained by applying builders on the tuple. The v in t^v stands for vanilla because no tuples have been used on t^v .

t^v is called a left-hand acceptable tuple for S and t^b a right-hand acceptable tuple.

We now define pointing.

Definition 4 (Pointing) For any sub-right-rule S , S pointed noted $S^\bullet$ is $S^\bullet = \text{Point}(S)$.

For a grammar $G = \{A_1 \rightarrow R_1, \dots, A_n \rightarrow R_n\}$, $G^\bullet = \{A_1 \rightarrow R_1, \dots, A_n \rightarrow R_n, A_1^\bullet \rightarrow R_1^\bullet, \dots, A_n^\bullet \rightarrow R_n^\bullet\}$.

For a context $\Gamma = (G, M)$, $\Gamma^\bullet = (G^\bullet, M^\bullet)$. For $A \rightarrow R \in G$, $M^\bullet(A) = M(A)$ and $M^\bullet(A^\bullet)(t^{b^\bullet}) = M(A)(\text{Unpoint}(t^{b^\bullet}, R))$ where $t^{b^\bullet}$ is a right acceptable tuple for $R^\bullet$. As it will be seen later, $M(A)(\text{Unpoint}(t^{b^\bullet}, R))$ is correctly defined.

Finally we define the size of a left-hand tuple: $|t^v|_z$

$$\begin{aligned} |t^v|_z &= \\ |t^v = Tp(i, t^{v'}) &\rightarrow |t^{v'}|_z \\ |t^v = Tp(t_1^v, \dots, t_n^v) &\rightarrow |t_1^v|_z + \dots + |t_n^v|_z \\ |t^v = Atom() &\rightarrow 1 \\ |t^v = Epsilon()|t^v = Marker(.) &\rightarrow 0 \end{aligned}$$

The theory to prove the three following theorems has now been established.

Theorem 1 (UnpointTp is an unpointing operation.) Let $\Gamma = (G, M)$ be a context, R a sub-right-rule appearing in G and t^v, t^b tuples.

$\Gamma^\bullet : R^\bullet \vdash (t^{v^\bullet}, t^{b^\bullet})$ if and only if $\text{Unpoint}(t^{v^\bullet}, R)$ is correctly defined, $\text{Unpoint}(t^{b^\bullet}, R)$ is correctly defined and $\Gamma : R \vdash (\text{UnpointTp}(t^{v^\bullet}, R), \text{UnpointTp}(t^{b^\bullet}, R))$.

Corollary 1 For a context $\Gamma = (G, M)$, $A \rightarrow R \in G$, $t^{b^\bullet}$ a tuple. If $t^{b^\bullet}$ is a right-hand acceptable tuple for $R^\bullet$. Then $M(A)(\text{Unpoint}(t^{b^\bullet}, R))$ is correctly defined. Therefore $M^\bullet(A^\bullet)$ is correctly defined.

Theorem 2 (UnpointTp respects the size of tuples) Let $\Gamma = (G, M)$ be a context, R a sub-right-rule appearing in G and $t^{v^\bullet}, t^{b^\bullet}$ tuples such that $\Gamma^\bullet : R^\bullet \vdash (t^{v^\bullet}, t^{b^\bullet})$. Then $\text{UnpointTp}(t^{v^\bullet}, R)|_z = |t^{v^\bullet}|_z$.

Theorem 3 (For n tuples pointed of size n there is one non pointed tuple)

Let $\Gamma = (G, M)$ be a context, R a sub-right-rule appearing in G and t^v, t^b tuples such that $\Gamma : R \vdash (t^v, t^b)$. Then $\{t^{v^\bullet} | \text{UnpointTp}(t^{v^\bullet}, R) = t^v \text{ and } t^v \text{ is left-hand acceptable for } R^\bullet\} = |t^v|_z$.

All three theorems will be proven by an induction on sub-right-rules.

But first we need to prove a small lemma.

Lemme 1 Let $\Gamma = (G, M)$ a context, R a sub-right-rule appearing in G and (t^v, t^b) . We have that:

$\Gamma^\bullet : R \vdash (t^v, t^b)$ if and only if $\Gamma : R \vdash (t^v, t^b)$.

Proof 1 The pointing operation create new productions rules but doesn't modify any production rule already present nor the mapping on non terminal present

before the pointing. In other words: $G \subset G^\bullet$ and $M \subset M^\bullet$. Moreover the proof tree for $\Gamma^\bullet : R \vdash (t^v, t^b)$ can't involve any pointed element because no inference rule allow that. Therefore $\Gamma^\bullet : R \vdash (t^v, t^b)$ and $\Gamma : R \vdash (t^v, t^b)$ have the same valid proof tree.

Proof 2 (Unpoint réalise bien une opération de dépointage) *Cas de bases:*

- $A = \text{Epsilon}()$ donc $A^\bullet = \text{Zero}()$, on ne peut pas avoir $\text{Zero}() \vdash t$ car $\text{Zero}()$ n'est pas associé à une règle d'inférence. D'un autre coté, $\text{Unpoint}(t, \text{Epsilon}())$ lève une exception (donc n'est pas correctement défini).
- $\text{Marker}()$ et $\text{Zero}()$ sont très similaires à $\text{Epsilon}()$.
- $A = \text{Atom}()$ donc $A^\bullet = \text{Atom}()$. On a donc $\text{Unpoint}(t, A) = t$. Si $\Gamma^\bullet : \text{Atom}() \vdash t$, alors on peut utiliser la seule règle d'inférence faisant intervenir l'atome pour en déduire que $t = \text{Atom}()$. Donc, en appliquant cette même règle d'inférence on en déduit que $G : \text{Atom}() \vdash t$. Le sens inverse est identique.

Cas rékursifs:

- $A = A_1 + \dots + A_n$, alors $A^\bullet = A_1^\bullet + \dots + A_n^\bullet$. On applique l'unique règle d'inférence associée à l'union, on en déduit que $t = (i, t')$ avec $1 \leq i \leq n$ et $\Gamma^\bullet : A_i^\bullet \vdash t'$. A apparait dans G , donc A_i apparait dans G , on peut donc utiliser notre induction. Par hypothèse d'induction, $\Gamma : A_i \vdash \text{Unpoint}(t', A_i)$ et donc, en appliquant la règle d'inférence pour l'union,

$$\Gamma : A \vdash (i, \text{Unpoint}(t', A_i)) = \text{Unpoint}((i, t'), A) = \text{Unpoint}(t, A)$$

. Le sens réciproque est identique.

- $A = A_1 \times \dots \times A_n$, alors $A^\bullet = A_1^\bullet \times A_2 \times \dots \times A_n + \dots + A_1 \times \dots \times A_{n-1} \times A_n^\bullet$. En appliquant la règle d'inférence pour l'union, on en déduit que $t = (i, t')$ avec $t' = (t_1, \dots, t_n)$. Puisque $\Gamma^\bullet : A_1 \times \dots \times A_i^\bullet \times \dots \times A_n \vdash (t_1, \dots, t_n)$, on peut cette fois ci appliquer la règle d'inférence pour le produit. Alors, pour $j \neq i$ $G^\bullet : A_j \vdash t_j$, or A_j apparait dans G car A apparait dans G , donc on peut appliquer notre lemme pour obtenir que $G : A_j \vdash t_j$. Par hypothèse d'induction, on sait aussi que $\Gamma : A_i \vdash \text{Unpoint}(t_i, A_i)$, on peut utiliser l'inférence pour le produit pour en déduire que

$$\Gamma : A_1 \times \dots \times A_n \vdash (t_1, \dots, \text{Unpoint}(t_i, A_i), \dots, t_n) = \text{Unpoint}(t, A)$$

Le sens réciproque est identique.

- Si A est une classe auxiliaire, la seule règle applicable est la simplification. Il existe donc un t et une règle R associée à A tel que $\Gamma^\bullet : A^\bullet \vdash B^\bullet(t)$ et $\Gamma^\bullet : R^\bullet \vdash t$. Par induction, $\Gamma : R \vdash \text{Unpoint}(t, R)$ et par simplification,

$$\Gamma : A \vdash B(\text{Unpoint}(t, R)) = B^\bullet(t) = \text{Unpoint}(B^\bullet(t), A)$$

Le sens inverse est identique.

Proof 3 (Unpoint respecte la taille des tuples) *Cas de bases:*

- $A = \text{Epsilon}()$, comme expliqué plus haut, ce cas ne peut pas se produire.
- Il en va de même pour $\text{Marker}()$ et $\text{Epsilon}()$.
- $A = \text{Atom}()$, alors $t = \text{Atom}()$, donc $t = \text{Unpoint}(t, A)$ et donc $|t|_z = \text{Unpoint}(t, A)_z$.

Cas récurifs:

- $A = A_1 + \dots + A_n$, donc $t = (i, t')$. Par induction, $|\text{Unpoint}(t', A_i)|_z = |t'|_z$. Or, $|(i, t')|_z = |t'|_z$ et $\text{Unpoint}((i, t'), A) = \text{Unpoint}(t', A_i)$, ce qui est suffisant pour conclure.
- A est une classe auxiliaire. On se donne t', B et R tel que: $t = (t', B(t'))$ et $\Gamma^\bullet : R^\bullet \vdash t'$. À nouveau, on peut conclure en appliquant l'induction.